

Anvesh Chandra Reddy 192472303

ITA0502 Computer

Vision

Slot -C

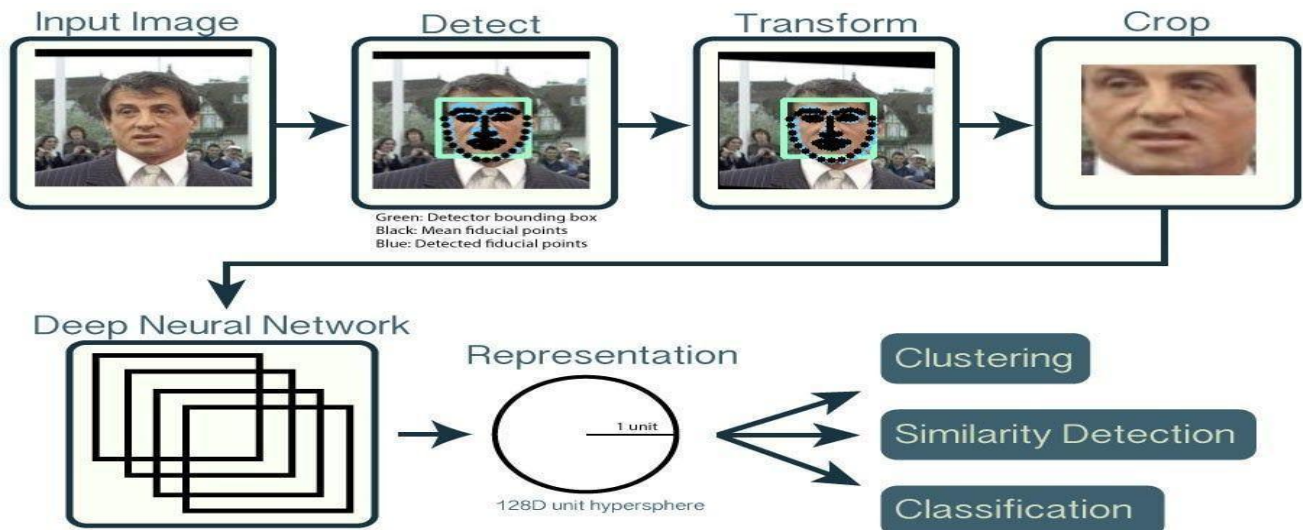
PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

Objective: Design an OpenCV-based system that continuously captures frames from a camera, detects multiple faces, recognizes registered persons, and displays their names with bounding boxes while maintaining real-time performance.

Overall Pipeline:

Camera → Frame Acquisition → Preprocessing → Face Detection → Face Feature Extraction → Recognition → Confidence Check → Visualization → Display





Explanation of Important Steps

BEGIN

1. INITIALIZE SYSTEM

- Load OpenCV library
- Load trained face detection model
- Load trained face recognition dataset/model
- Set recognition threshold
- Set frame-skip value if required

2. OPEN CAMERA

```

camera ← Open webcam/CCTV stream
IF camera cannot be opened THEN
    Display "Camera Error"
    EXIT
END IF
  
```

3. START REAL-TIME LOOP

```

WHILE camera is open DO
    3.1 CAPTURE FRAME
    frame ← Read frame from camera
    IF frame is empty THEN
        BREAK
    END IF
    3.2 OPTIMIZE FRAME
    Resize frame to suitable resolution
    Convert frame to grayscale
    Apply noise reduction if required
    3.3 DETECT FACES
    faces ← Detect all faces in frame
    FOR each detected face DO
  
```

```

x, y, width, height ← face coordinates
3.4 EXTRACT FACE
face_region ← Crop face from frame
Resize face_region to required size
3.5 FEATURE EXTRACTION
features ← Extract facial features
OR
features ← Generate face embedding
3.6 RECOGNITION
Compare features with trained dataset
best_match ← Find closest registered person
confidence ← Calculate similarity/confidence
3.7 IDENTITY DECISION
IF confidence satisfies recognition threshold THEN
    identity ← Registered person's name
ELSE
    identity ← "Unknown"
END IF
3.8 DRAW RESULT
Draw bounding box around face
Display identity label above bounding box
END FOR
3.9 DISPLAY FRAME
Show processed frame
3.10 CHECK EXIT CONDITION
key ← Read keyboard input
IF key is ESC OR 'q' THEN
    BREAK
END IF
END WHILE

```

4. RELEASE RESOURCES

```

Release camera
Close all OpenCV windows
Free memory/resources
END

```

1. Image Acquisition

The camera continuously captures frames.

Camera → Frame 1 → Frame 2 → Frame 3 → ...

OpenCV's VideoCapture can be used to access the webcam or CCTV stream.

2. Preprocessing

Each frame can be:

- Resized to reduce computation.
- Converted to grayscale for lightweight face detection.
- Filtered to reduce camera noise.

This reduces processing time and improves detection quality.

3. Multiple Face Detection

The detector searches the complete frame and returns multiple bounding boxes.

For example:

Face 1 → Student A

Face 2 → Student B

Face 3 → Unknown

Face 4 → Student C

The system processes each detected face independently.

4. Feature Extraction

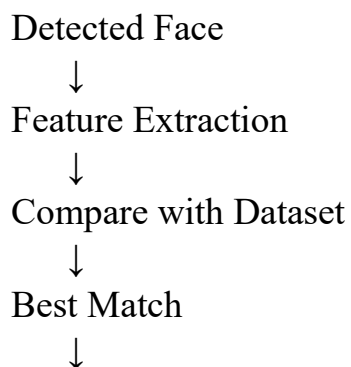
The detected face is converted into useful numerical features.

For a lightweight OpenCV implementation, **LBPH** can be used.

For a more advanced system, facial embeddings can be generated using a deep facerecognition model.

5. Recognition

The extracted features are compared with the trained dataset.



Confidence Check



Name / Unknown

6. Confidence Threshold

A threshold is important to reduce false recognition.

IF confidence is sufficient

→ Recognized Person

ELSE

→ Unknown

The exact threshold depends on the selected recognition method.

7. Visualization

The final frame displays:

- Face bounding box.
- Person's identity.
- Unknown label when no reliable match exists.

8. Real-Time Optimization To

minimize processing delay:

- Resize input frames.
- Process a smaller frame resolution.
- Skip some frames if necessary.
- Detect faces periodically.

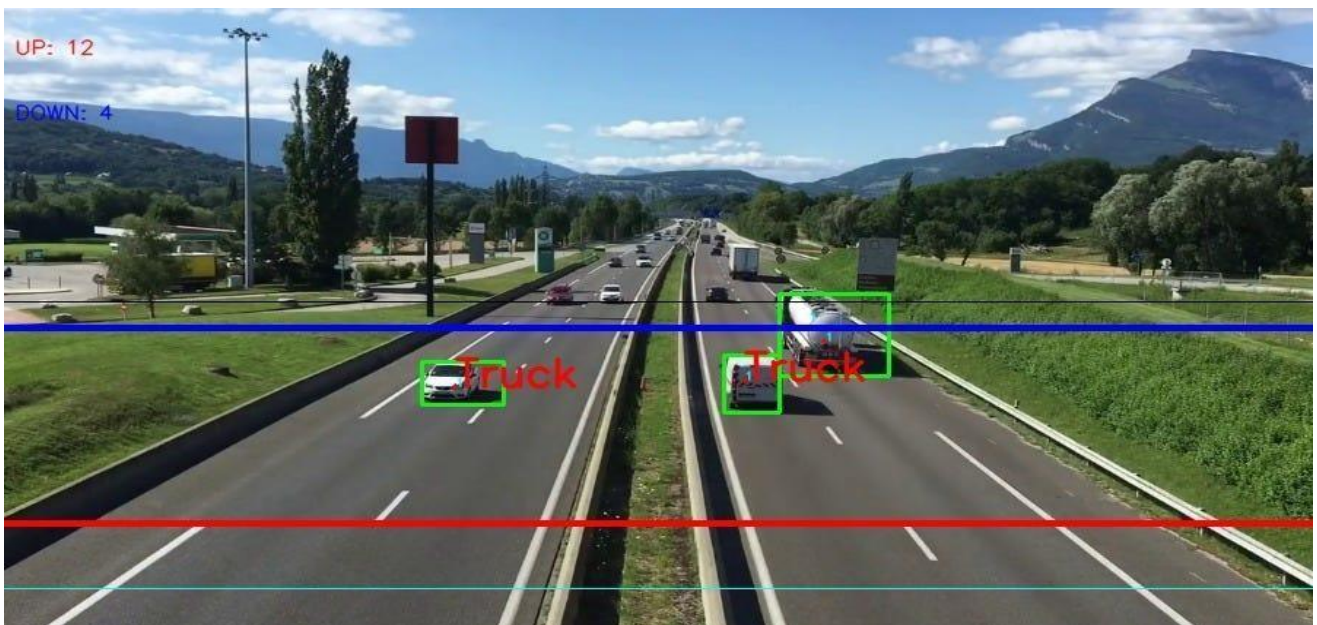
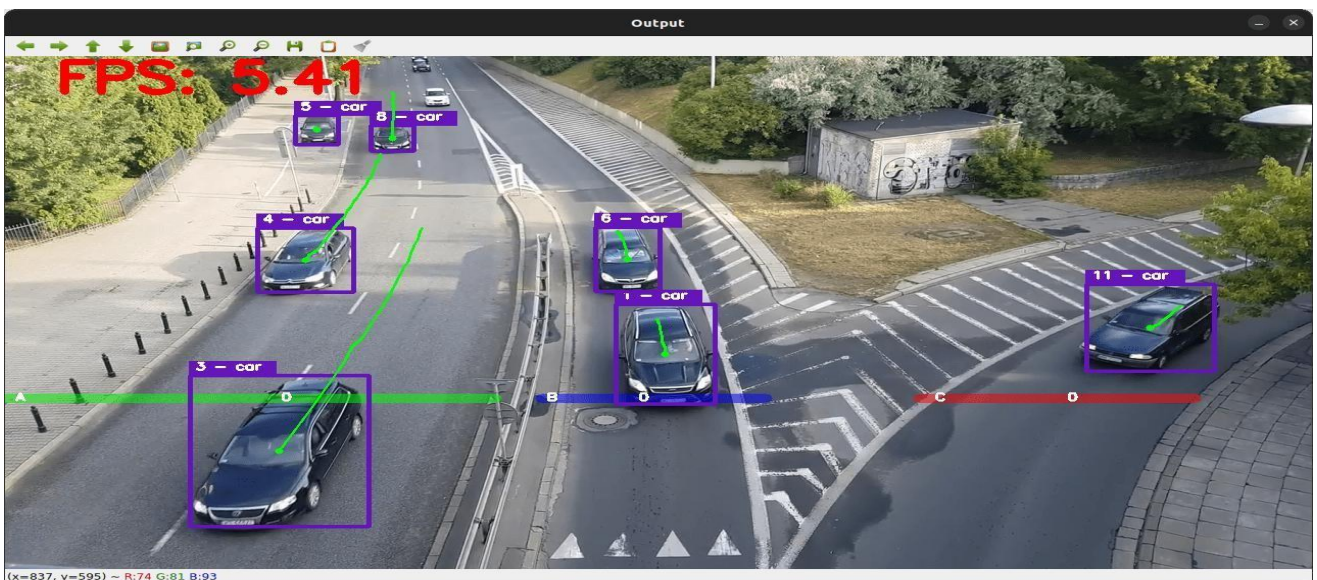
2. Vehicle Detection, Tracking and Counting

Objective

Design an OpenCV-based traffic monitoring system that detects multiple vehicles, tracks them across frames, counts each vehicle when it crosses a predefined line, and prevents double counting.

Overall Pipeline

Video → Preprocessing → ROI → Vehicle Detection → Tracking → ID Assignment → Line Crossing → Counting → Visualization



BEGIN

1. INITIALIZE SYSTEM

Load OpenCV library
Load vehicle detection method/model
Initialize object tracker
Set Region of Interest (ROI)
Set virtual counting line
vehicle_count $\leftarrow$ 0
counted_IDs $\leftarrow$ empty set
next_ID $\leftarrow$ 1

2. OPEN VIDEO SOURCE

video $\leftarrow$ Open live camera/video stream
IF video cannot be opened THEN
 Display "Video Error"
 EXIT
END IF

3. START PROCESSING LOOP

WHILE video is open DO

3.1 READ FRAME

frame $\leftarrow$ Read frame
IF frame is empty THEN
 BREAK
END IF

3.2 PREPROCESS FRAME

Resize frame if required
Apply Gaussian/median filtering
Define/use ROI
roi_frame $\leftarrow$ Extract ROI from frame

3.3 DETECT VEHICLES

detections $\leftarrow$ Detect vehicles in ROI
FOR each detection DO
 Obtain bounding box
 Obtain vehicle class
 Obtain confidence
 IF confidence is below threshold THEN

```

    Ignore detection
  END IF
END FOR
3.4 UPDATE TRACKERS
Send valid detections to tracker
tracker updates vehicle positions
FOR each tracked vehicle DO
  Obtain tracking ID
  Obtain current bounding box
  Calculate center point
END FOR
3.5 CHECK VIRTUAL LINE CROSSING
FOR each tracked vehicle DO
  current_position ← Current center point
  previous_position ← Previous center point
  IF vehicle moved from one side of virtual line to the other THEN
    IF tracking ID is NOT in counted_IDs THEN
      vehicle_count ← vehicle_count + 1
      Add tracking ID to counted_IDs
    END IF
  END IF
  Store current position as previous position
END FOR
3.6 VISUALIZATION
Draw ROI boundary
Draw virtual counting line
FOR each tracked vehicle DO
  Draw bounding box
  Display tracking ID
  Display vehicle class if required
  Draw center point
END FOR
Display vehicle_count
3.7 DISPLAY FRAME
Show processed frame
3.8 CHECK EXIT CONDITION
key ← Read keyboard input
IF key is ESC OR 'q' THEN
  BREAK
END IF
END WHILE

```


4. RELEASE RESOURCES

Release video source
Destroy all OpenCV windows
Release tracker resources

END

1. Video Acquisition

The system continuously reads frames from:

- CCTV camera
- Webcam
- Traffic surveillance camera
- Recorded traffic video

Camera



Frame 1



Frame 2



Frame 3



If the video source cannot be opened, the program terminates safely.

2. Preprocessing The

input frame can be:

- Resized.
- Smoothed using Gaussian/median filtering.
- Restricted to a **Region of Interest**.

The ROI prevents the system from processing irrelevant areas such as buildings, sky, and sidewalks.

3. Vehicle Detection

The detector identifies vehicles inside the ROI.

Possible approaches include:

- Background subtraction + contours for a simple fixed-camera system.
- OpenCV DNN/object detection for more reliable vehicle detection.

The detector returns:

Bounding Box

Vehicle Class Confidence

Score

Example:

Car → Confidence 95%

Bus → Confidence 91%

Truck → Confidence 88%

Low-confidence detections are ignored.

4. Vehicle Tracking

Detection alone is not sufficient because the same vehicle appears in many consecutive frames.

Tracking assigns an ID:

Frame 1 → Car → ID 1

Frame 2 → Car → ID 1

Frame 3 → Car → ID 1

Frame 4 → Car → ID 1

Therefore, the system understands that these detections belong to the **same vehicle**.

6. Line-Crossing Logic

Suppose the counting line is at position Y.

If:

Previous Y < Line Y

AND

Current Y >= Line Y then the

vehicle has crossed the line.

The counter is increased:

vehicle_count = vehicle_count + 1

7. Avoiding Double Counting

This is one of the most important constraints.

Maintain a set:

counted_IDs = {1, 4, 7}

Before increasing the count: IF

vehicle_ID NOT IN counted_IDs

count = count + 1

 Add vehicle_ID to counted_IDs

END IF

Therefore:

Vehicle ID 1 crosses line → Count = 1

Vehicle ID 1 appears again → Do NOT count

Vehicle ID 1 appears again → Do NOT count

This prevents duplicate counting.

8. Multiple Vehicles

Each vehicle has its own:

- Bounding box
- Tracking ID
- Center position
- Previous position
- Counting status

9. Visualization

The output frame displays:

```
+-----+
|          TRAFFIC CAMERA          |
|                                  |
| [Car] ID: 1                      |
|   ↓                             |
|=====|
|          COUNTING LINE          |
|                                  |
| [Truck] ID: 2                    |
|                                  |
| Vehicle Count: 12                |
+-----+ The
```

following information can be displayed:

- Bounding boxes.
- Vehicle IDs.
- Vehicle classes.
- Virtual line.
- ROI.
- Total vehicle count.

10. Real-Time Optimization To

maintain near real-time performance:

- Resize frames.
- Process only the ROI.
- Use efficient detection models.
- Track vehicles between detection frames.
- Skip unnecessary frames when appropriate.
- Remove expired tracking IDs.
- Avoid repeatedly detecting the entire image.